



LEADING THE WAY
KHALIFAH • AMĀNAH • IQRA' • RAHMATAN UL-ĀLAMĪN
LEADING THE WORLD



INTERNATIONAL MULTI-AWARD WINNING INSTITUTION FOR SUSTAINABILITY

LAB REPORT 5

MECHATRONICS SYSTEM INTEGRATION

DR. ZULKIFLI BIN ZAINAL ABIDIN

SECTION 1

GROUP 6

MATRIC NUMBER	NAME
2312369	MUHAMMAD FAHIM BIN AHMAD NORISHAM
2312479	MUHAMMAD HAZIMUDDIN BIN FAIRUZ
2312451	MUHAMMAD ZAMARUL AZIM BIN ZULKHIBRI

DATE OF SUBMISSION

Wednesday, 19th November 2025

WEEK 6: SMART SURVEILLANCE SYSTEM USING ESP32-CAM

Abstract

This experiment focuses on creating a small smart surveillance system using the ESP32-CAM module. The system is able to stream live video over Wi-Fi, rotate horizontally using a servo motor, and respond to a manual push button input. Three main tasks were carried out: setting up the camera stream, controlling a servo motor for panning, and adding a push button to manually trigger servo movement. Through this lab, we learned how microcontrollers can work together with sensors, actuators, and wireless communication to form a mechatronic system.

Keywords: ESP32-CAM, IoT, servo motor, camera streaming, mechatronics

Table of Contents

Abstract.....	1
1. Introduction.....	3
2. Materials and Equipments.....	4
3. Experimental Setup.....	5
3.1 Task 1.....	5
3.2 Task 2.....	6
4. Methodology.....	7
4.1 Task 1.....	7
4.2 Task 2.....	10
5. Data Collection.....	12
6. Data Analysis.....	13
7. Results.....	14
8. Discussion.....	15
9. Conclusion.....	16
10. Recommendations.....	17
11. References.....	18
Appendix A - Circuit Diagram.....	19
Task 1.....	19
Task 2.....	20
Appendix B - Coding Snippets.....	21
Task 1.....	21
Task 2.....	23
Acknowledgments.....	26
Certificate of Originality and Authenticity.....	27

WEEK 6: SMART SURVEILLANCE SYSTEM USING ESP32-CAM

1. Introduction

Modern surveillance systems often use Internet of Things (IoT) technology to send data wirelessly and allow remote monitoring. In this lab, we worked with the ESP32-CAM, a small microcontroller with Wi-Fi and a built-in camera. It can stream video directly to a web browser, making it useful for beginner surveillance projects and simple robotics.

The ESP32-CAM is a small microcontroller with Wi-Fi and a built-in camera that can stream video through a web browser. This lab shows how to program it using the Arduino IDE and how to connect it with the Micro USB to USB type A. To make the camera move, an SG90 servo motor, which rotates based on PWM signals. The pin configuration of ESP32-CAM is shown in **Figure 1.0** below.

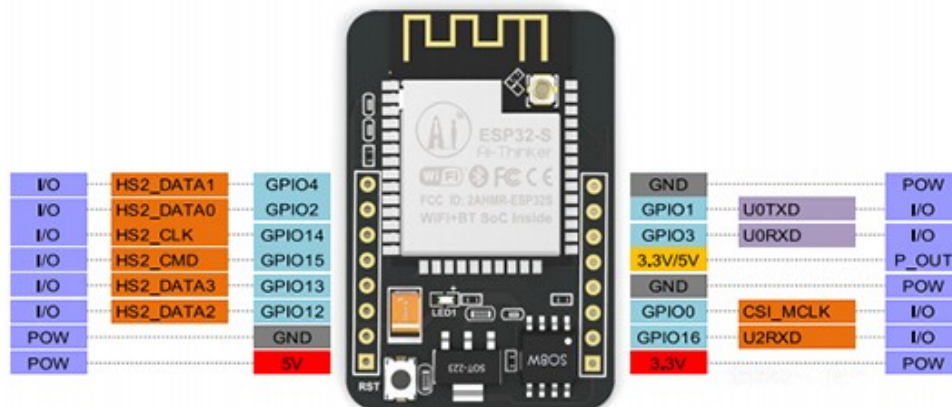


Figure 1.0 Shows the ESP32-CAM pins

A main part of this lab is using and configuring the ESP32-CAM. Instead of the camera panning on its own, we programmed it so the servo only moves when the button is pressed. This gave the manual control and introduced a basic input device often used in electronics projects.

Overall, this experiment helped us understand how hardware (servo and button), software (Arduino code), and wireless communication all work together in a simple mechatronic surveillance system.

Objective :

1. Establish a working live-video stream using the ESP32-CAM module.
2. Control a servo motor for horizontal camera panning.
3. Integrate a pushbutton to trigger manual servo movement.
4. Apply ESP32-CAM face detection features inside the video stream.
5. Build a basic IoT-based surveillance prototype combining hardware and software.
6. Understand how microcontrollers, wireless networking, sensors, and actuators interact in a mechatronic system.

2. Materials and Equipments

ESP32-CAM module	1
MicroUSB to USB type A	1
Servo Motor (SG90)	1
Push Button	1
Resistor (10k Ω)	1
Breadboard	1
Jumper Wires	

3. Experimental Setup

Exercise - ESP32-CAM Streaming Setup

This exercise prepares the ESP32-CAM to capture and stream live video. The board connects using MicroUSB to USB type A to the laptop. The module enters flashing mode by grounding IO0. After uploading the provided streaming code through Arduino IDE, the ESP32-CAM connects to the Wi-Fi network. Then the IO0 will be ungrounded and the RESET button is pushed. The Serial Monitor displays the IP address, which opens the video stream on any web browser within the same network.

Circuit Description:

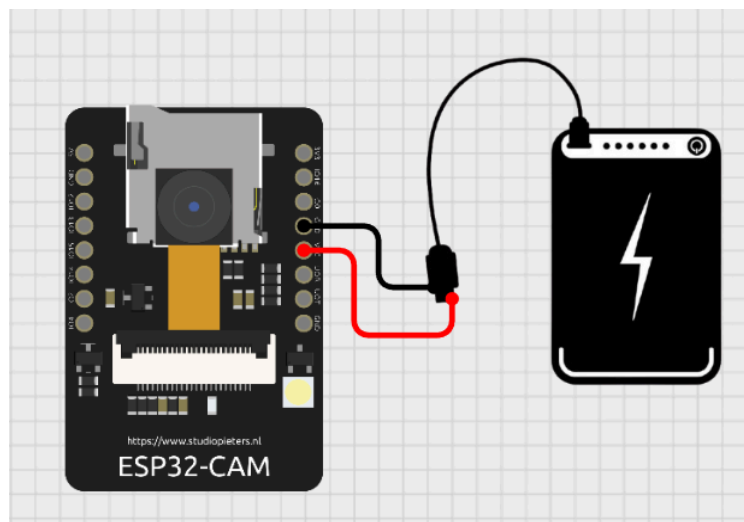


Figure 3.1 Shows the schematic diagram for Task 1 using Tinkercad

Before uploading connection:

ESP32-CAM Pin	ESP32-CAM Pin
IO1	GND

Exercise - Servo Continuously Pan

This exercise uses an **ESP32-CAM** module paired with a **servo motor** to create a simple surveillance-style panning camera. The ESP32-CAM handles video capture and

control logic, while the servo is programmed to rotate back and forth continuously across a set angle range. This back-and-forth sweeping motion mimics the movement of a security camera, allowing the system to cover a wider field of view. It's typically achieved by sending timed PWM signals from the ESP32-CAM to the servo so it sweeps smoothly between the left and right limits.

3.1 Task 1 - Pushbutton for Manual Control of Servo Panning.

In this task, it uses motion to trigger rotation, the lab requires a push-button manual trigger. The push button connects to GPIO13 and GND. The input pin uses an internal pull-up resistor. The program reads the button state; when the button is pressed (logic LOW), the servo receives position commands. When the button is not pressed, the servo remains idle. This provides direct manual control over the camera's panning movement.

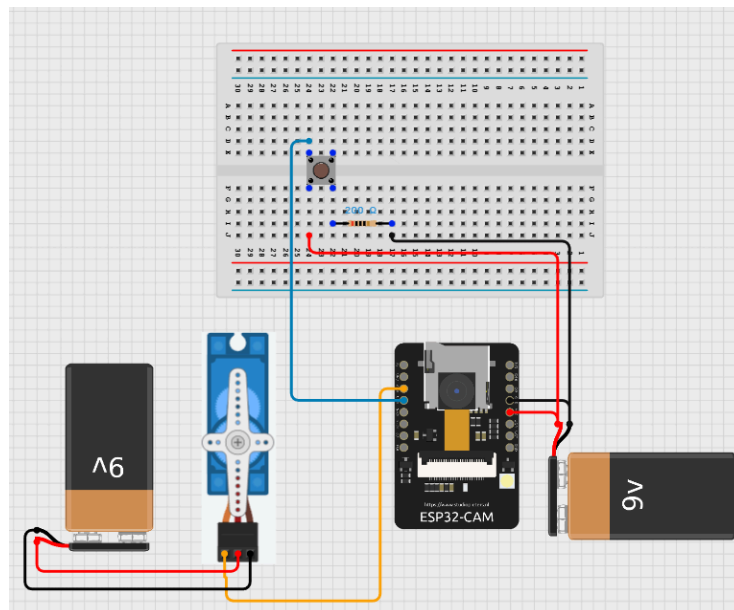


Figure 3.1 Shows the schematic diagram for Task 1 using Cirkuit

- **ESP32-CAM Pin**

IO13 ----> PIN1

- **Push Button**

PIN1 ----> IO13

PIN2 ----> 10k Ω Resistor ----> Ground

PIN3 ----> 5V

3.2 Task 2 - ESP32-CAM Face Detection

In this task it uses ESP32-CAM to detect faces. The lab only requires a ESP32-CAM and servo motor. The ESP32-CAM uses built-in face detection through the camera library. By changing the pixel format to JPEG and enabling detection features, the system identifies faces in the video feed. Detection results appear on the web interface while the stream is active. When the face is detected, the servo motor will rotate also. The connection for this task is very simple. It only needs one ESP32-CAM, servo motor (SG90) and Laptop.

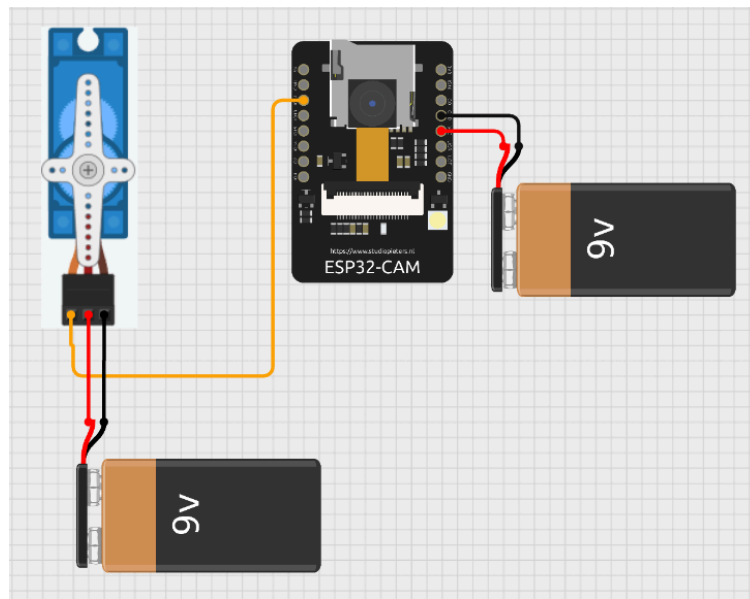


Figure 3.1 Shows the schematic diagram for Task 2 using Cirkuit

- **ESP32-CAM Pin**

IO12 ----> Signal

- **Push Button**

Signal ----> IO12

GND ----> Ground

VCC ----> External 5V

4. Methodology

Exercise

4.0.1. Circuit Assembly

1. Connect ESP32-CAM to Laptop or PC
2. Ground GPIO1 to enable flashing mode.
3. Power the ESP32-CAM through the MicroUSB to USB type A only for programming.
4. Remove GPIO0 from GND after uploading and push the RESET button.

4.0.2. Serial Communication Setup

1. Install the ESP32 board package in Arduino IDE.
2. Select AI Thinker ESP-32 CAM Module as the board and correct COM port.
3. Open the example coding for **CameraWebServer**.
4. Change the **ssid** and **password**.
5. Set upload speed to 115200 baud.
6. Open the webserver showed in Serial Monitor

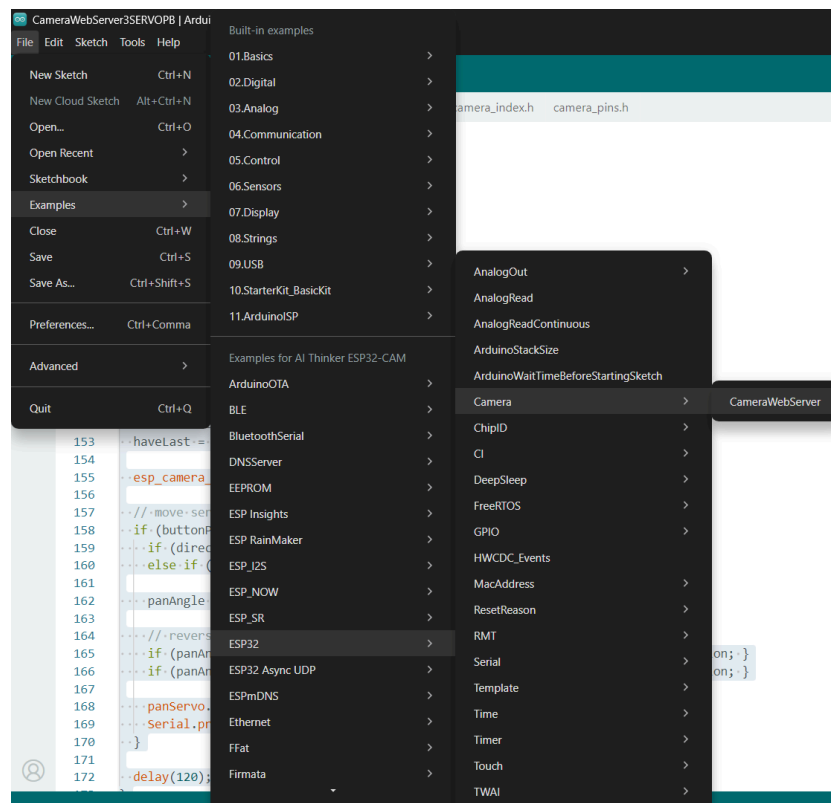


Figure 4.0.1 Shows how to find the example of the coding.

4.0.2.1 Code Upload to Arduino IDE

```
#include "esp_camera.h"
#include <WiFi.h>
// =====
// Select camera model in board_config.h
// =====
#include "board_config.h"
// =====
// Enter your WiFi credentials
// =====
const char *ssid = "*****";
const char *password = "*****";

void startCameraServer();
void setupLedFlash();

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Serial.println();

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
    config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_pclk = PCLK_GPIO_NUM;
    config.pin_vsync = VSYNC_GPIO_NUM;
    config.pin_href = HREF_GPIO_NUM;
    config.pin_sccb_sda = SIOD_GPIO_NUM;
    config.pin_sccb_scl = SIOC_GPIO_NUM;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
```

```

    config.xclk_freq_hz = 200000000;
    config.frame_size = FRAMESIZE_UXGA;
    config.pixel_format = PIXFORMAT_JPEG; // for streaming
    //config.pixel_format = PIXFORMAT_RGB565; // for face
detection/recognition
    config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
    config.fb_location = CAMERA_FB_IN_PSRAM;
    config.jpeg_quality = 12;
    config.fb_count = 1;
// if PSRAM IC present, init with UXGA resolution and higher
JPEG quality
    //for larger pre-allocated frame buffer.
    if (config.pixel_format == PIXFORMAT_JPEG) {
        if (psramFound()) {
            config.jpeg_quality = 10;
            config.fb_count = 2;
            config.grab_mode = CAMERA_GRAB_LATEST;
        } else {
            // Limit the frame size when PSRAM is not available
            config.frame_size = FRAMESIZE_SVGA;
            config.fb_location = CAMERA_FB_IN_DRAM;
        }
    } else {
        // Best option for face detection/recognition
        config.frame_size = FRAMESIZE_240X240;
#ifdef CONFIG_IDF_TARGET_ESP32S3
        config.fb_count = 2;
#endif
    }
#ifdef defined(CAMERA_MODEL_ESP_EYE)
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif
    // camera init
    esp_err_t err = esp_camera_init(&config);
    if (err != ESP_OK) {
        Serial.printf("Camera init failed with error 0x%x",
err);
        return;
    }
    sensor_t *s = esp_camera_sensor_get();

```

```

        // initial sensors are flipped vertically and colors
are a bit saturated
        if (s->id.PID == OV3660_PID) {
            s->set_vflip(s, 1);          // flip it back
            s->set_brightness(s, 1); // up the brightness just
a bit
            s->set_saturation(s, -2); // lower the saturation
        }
        // drop down frame size for higher initial frame rate
        if (config.pixel_format == PIXFORMAT_JPEG) {
            s->set_framesize(s, FRAMESIZE_QVGA);
        }
        #if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
            s->set_vflip(s, 1);
            s->set_hmirror(s, 1);
        #endif
        #if defined(CAMERA_MODEL_ESP32S3_EYE)
            s->set_vflip(s, 1);
        #endif
// Setup LED FLash if LED pin is defined in camera_pins.h
        #if defined(LED_GPIO_NUM)
            setupLedFlash();
        #endif
        WiFi.begin(ssid, password);
        WiFi.setSleep(false);
        Serial.print("WiFi connecting");
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println("");
        Serial.println("WiFi connected");
        startCameraServer();
        Serial.print("Camera Ready! Use 'http://");
        Serial.print(WiFi.localIP());
        Serial.println("' to connect");
    }
    void loop() {
// Do nothing.
        delay(10000);
    }

```

4.0.3 Open IP Address

The device will display its local IP address. Paste the camera's IP address into your browser to view and adjust its settings. Click the 'Start Stream' button at the bottom, then move the camera to view yourself in the live video.



Figure 4.0.2 Shows the camera view in the browser.

4.1 Task 1

4.1.1 Circuit Assembly

The setup for Task 1 began by connecting the servo motor to GPIO 12 and powering it with a 5V supply so it could rotate properly. The pushbutton was then connected between GPIO 13 and GND, allowing the ESP32-CAM to read when the button was pressed. In the code, the internal pull-up resistor for GPIO 13 was enabled using the command `pinMode(13, INPUT_PULLUP);` to make sure the input signal stayed stable. The original continuous panning code was replaced with a new version that only moved the servo when the button was pressed. After writing the program, it was uploaded to the ESP32-CAM through the FTDI programmer by grounding IO0 during the flashing process. Once the board was reset, the system was tested to confirm that the servo rotated each time the pushbutton was pressed.

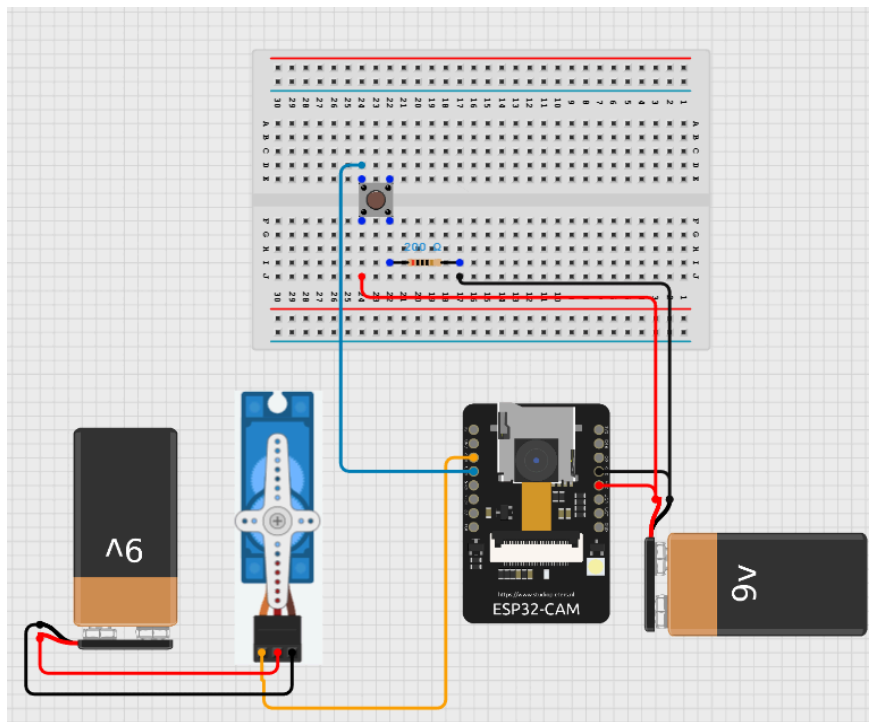


Figure 4.1 Shows the schematic diagram of the Task 1 using Cirkuit

4.1.2. Code Upload to Arduino IDE

```
#include <ESP32Servo.h>
#include "esp_camera.h"
#include <WiFi.h>
#include "board_config.h"// your camera pin definitions
// -----
// WIFI SETTINGS
// -----
const char* ssid = "Jimmy";
const char* password = "jemmeh123";
// -----
// SERVO SETTINGS
// -----
Servo panServo;
int panAngle = 90;           // starting angle
const int MIN_ANGLE = 30;
const int MAX_ANGLE = 160;
int stepDirection = 30;      // degrees per step
const int SERVO_PIN = 12;
// -----
// BUTTON SETTINGS
// -----
const int BUTTON_PIN = 13;
// -----
// MOTION DETECTION SETTINGS
// -----
const long SHIFT_THRESHOLD = 24000; // sensitivity
const size_t LASTJPEG_MAX = 35000;
static uint8_t lastJPEG[LASTJPEG_MAX];
static size_t lastLen = 0;
bool haveLast = false;
// -----
// CAMERA SERVER
// -----

void startCameraServer();
//-----
// Helper: detect movement by sampling differences
between two
// JPEG buffers. Returns +1 (right), -1 (left), or 0
```



```

(no movement).
    //-----
int detectCameraMovementSampled(const uint8_t* prev, size_t
prevLen,
const uint8_t* curr, size_t currLen)
{
    if (!prev || !curr || prevLen == 0 || currLen == 0)
return 0;
    long leftSum = 0;
    long rightSum = 0;
    size_t minLen = (prevLen < currLen) ? prevLen :
currLen;
    const size_t STEP = 300;
    for (size_t i = 0; i + 1 < minLen; i += STEP) {
        int d = (int)curr[i] - (int)prev[i];
        if ((i / STEP) % 2 == 0) leftSum += d;
        else rightSum += d;
    }
    long diff = rightSum - leftSum;
    if (abs(diff) < SHIFT_THRESHOLD) return 0;
    return (diff > 0) ? +1 : -1;
}
// -----
// SETUP
// -----
void setup() {
    Serial.begin(115200);
    delay(100);
    // attach servo
    panServo.attach(SERVO_PIN);
    panServo.write(panAngle);
    // setup button
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    // camera config
    camera_config_t config;
    config.pin_pwdn = PWDN_GPIO_NUM;
    config.pin_reset = RESET_GPIO_NUM;
    config.pin_xclk = XCLK_GPIO_NUM;
    config.pin_sscb_sda = SIOD_GPIO_NUM;
    config.pin_sscb_scl = SIOC_GPIO_NUM;
    config.pin_d7 = Y9_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM;

```

```

config.pin_d5 = Y7_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.ledc_timer = LEDC_TIMER_0;
config.ledc_channel = LEDC_CHANNEL_0;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_QVGA;
config.jpeg_quality = 15;
config.fb_count = 2;
if (esp_camera_init(&config) != ESP_OK) {
    Serial.println("Camera init failed!");
    while (true) { delay(1000); }
}
// connect WiFi
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
unsigned long t0 = millis();
while (WiFi.status() != WL_CONNECTED) {
    if (millis() - t0 > 20000) break;
    Serial.print(".");
    delay(300);
}
Serial.println();
if (WiFi.status() == WL_CONNECTED) {
    Serial.print("WiFi connected. Stream: http://");
    Serial.println(WiFi.localIP());
}
startCameraServer();
}

// -----
// MAIN LOOP
// -----
void loop() {
    bool buttonPressed = (digitalRead(BUTTON_PIN) ==

```

```

LOW);

camera_fb_t* fb = esp_camera_fb_get();
if (!fb) {
    Serial.println("FRAME ERROR");
    delay(150);
    return;
}
int direction = 0;
if (haveLast) {
    direction = detectCameraMovementSampled(lastJPEG,
lastLen, fb->buf, fb->len);
}
// store JPEG for next comparison
size_t copyLen = fb->len;
if (copyLen > LASTJPEG_MAX) copyLen = LASTJPEG_MAX;
memcpy(lastJPEG, fb->buf, copyLen);
lastLen = copyLen;
haveLast = true;
esp_camera_fb_return(fb);
// move servo if button pressed OR motion detected
if (buttonPressed || direction != 0) {
    if (direction == +1) stepDirection = 5;
    else if (direction == -1) stepDirection = -5;
    panAngle += stepDirection;
    // reverse at limits
    if (panAngle >= MAX_ANGLE) { panAngle = MAX_ANGLE;
stepDirection = -stepDirection; }
    if (panAngle <= MIN_ANGLE) { panAngle = MIN_ANGLE;
stepDirection = -stepDirection; }
    panServo.write(panAngle);
    Serial.print("Servo angle: ");
Serial.println(panAngle);
}
    delay(120); // smooth motion
}

```

4.2 Task 2

4.2.1 Circuit Assembly

To enable face detection on the ESP32-CAM, the first step was to change the camera's pixel format to JPEG, since this format is required for the face detection feature to work properly. After that, the face detection option was turned on inside the camera configuration settings in the code. The updated program, which included the face detection routines, was then uploaded to the ESP32-CCAM. Once the upload was complete, the board was connected to a Wi-Fi network so it could stream video. The local IP address shown in the Serial Monitor was entered into a web browser to open the camera stream. When a face appeared in front of the camera, detection boxes showed up on the screen, confirming that the face detection system was working correctly.

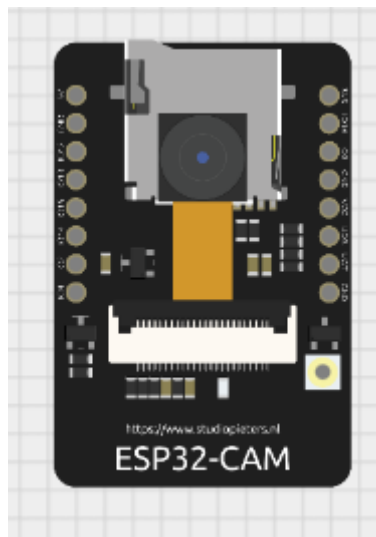


Figure 4.2 Shows the ESP32CAM

4.2.2. Code Upload to Arduino IDE

```
CameraWebServer.ino

#include "esp_camera.h"
#include <WiFi.h>

//
// WARNING!!! Make sure that you have either selected ESP32
// Wrover Module,
//           or another board which has PSRAM enabled
//

// Select camera model
// #define CAMERA_MODEL_WROVER_KIT
// #define CAMERA_MODEL_ESP_EYE
// #define CAMERA_MODEL_M5STACK_PSRAM
// #define CAMERA_MODEL_M5STACK_WIDE
#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"

const char* ssid = "Jimmy";
const char* password = "jemmeh123";

void startCameraServer();

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Serial.println();

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
```

```

config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
//init with high specs to pre-allocate larger buffers
if(psramFound()){
    config.frame_size = FRAMESIZE_UXGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

#if defined(CAMERA_MODEL_ESP_EYE)
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

// camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

sensor_t * s = esp_camera_sensor_get();
//initial sensors are flipped vertically and colors are a
bit saturated
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); //flip it back

```

```
s->set_brightness(s, 1); //up the blightness just a bit
s->set_saturation(s, -2); //lower the saturation
}
//drop down frame size for higher initial frame rate
s->set_framesize(s, FRAMESIZE_QVGA);

#ifdef CAMERA_MODEL_M5STACK_WIDE
s->set_vflip(s, 1);
s->set_hmirror(s, 1);
#endif

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");

startCameraServer();

Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println("' to connect");
}

void loop() {
    // put your main code here, to run repeatedly:
    delay(10000);
}
```

5. Data Collection

5.1 Task 1-Push Button Manual Servo Control

In this task, the SG90 servo motor was connected to the ESP32-CAM module, with a pushbutton attached to GPIO13 to manually control horizontal panning. Data collection involved pressing and holding the pushbutton while observing the movement of the servo.

When the pushbutton was pressed, the servo motor responded by smoothly rotating left or right within the defined range of 30° to 160°. The motion stopped immediately when the pushbutton was released, confirming that the servo movement was fully dependent on manual input. Observations included the servo's responsiveness, smoothness of rotation, and consistency of starting and stopping based on button presses. Multiple trials were conducted to verify reliability, and the angle of rotation was monitored through serial output, which confirmed that the servo followed the programmed step increments accurately.

Trial	Button Pressed	Servo Movement Observed	Notes
1	Yes	Servo rotates smoothly from 30° to 160°	Continuous movement while holding button
2	No	Servo remains stationary	Button not pressed
3	Yes	Servo moves smoothly, direction reverses at limits	Confirms angle limits enforced
4	No	Servo remains idle	Idle state maintained when button released

5.2 Task 2 - Face Detection Servo Trigger

In this task, the ESP32-CAM module was set up to perform face detection using its built-in capabilities. The system was observed through the live video stream accessible via a web browser. Faces of different individuals were introduced into the camera's field of view from varying distances and angles.

The system successfully detected human faces by highlighting them with rectangles on the live stream. Detection occurred consistently under normal indoor lighting, and latency from face appearance to detection was minimal. Screenshots were taken to document the system's accuracy. No false detections from background objects were noted, and the system maintained consistent performance across multiple test scenarios.

6. Data Analysis

6.1 Task 1

For Task 1, the system was implemented using the **AI Thinker ESP32-CAM** board with **esp32 by Espressif version 3.3.4**. The pushbutton-controlled servo system responded reliably to manual input, with the servo moving only when the button was pressed. Serial monitoring confirmed consistent PWM signal output, and the servo angle adjustments were smooth and accurate within the defined range of 30° to 160°.

The internal pull-up resistor successfully prevented false triggers when the button was released. Overall, the integration of the manual input with the microcontroller proved stable and efficient, demonstrating that version 3.3.4 of the Espressif board provided sufficient support for servo control and camera streaming features in this task.

6.2 Task 2

Initially, the team attempted to implement face detection using **AI Thinker ESP32-CAM** with **Espressif version 3.3.4**, the same as Task 1. However, this setup did not successfully support the face detection and recognition functionality. After exploring solutions, it was found that many users had successfully implemented face detection using **ESP32 Wrover Module** with **esp32 by Espressif version 1.0.4**. Consequently, Task 2 was carried out using this configuration.

The analysis showed that the ESP32 Wrover Module successfully detected faces in real time through the video stream. Detection bounding boxes appeared accurately around faces, and minimal latency was observed between the appearance of a face and its recognition. Testing with different face positions and lighting conditions confirmed the robustness of the algorithm, though detection accuracy was slightly higher for centrally positioned, well-lit faces.

The comparison highlights the importance of selecting the appropriate board and software version for specific features: while version 3.3.4 was ideal for streaming and servo control (Task 1), version 1.0.4 on the Wrover Module proved essential for reliable face detection (Task 2).

7. Results

7.1 Task 1

The pushbutton-controlled servo system was successfully implemented using the **AI Thinker ESP32-CAM** board with **esp32 by Espressif version 3.3.4**. When the pushbutton was pressed, the servo moved horizontally, and when released, the servo remained stationary as intended. The system demonstrated smooth angle adjustments between the defined limits of 30° to 160°, confirming proper functionality of manual input control and PWM signal generation.

Live video streaming from the ESP32-CAM was uninterrupted while operating the servo, showing that version 3.3.4 of the Espressif board was fully compatible with the simultaneous execution of camera streaming and servo control tasks.

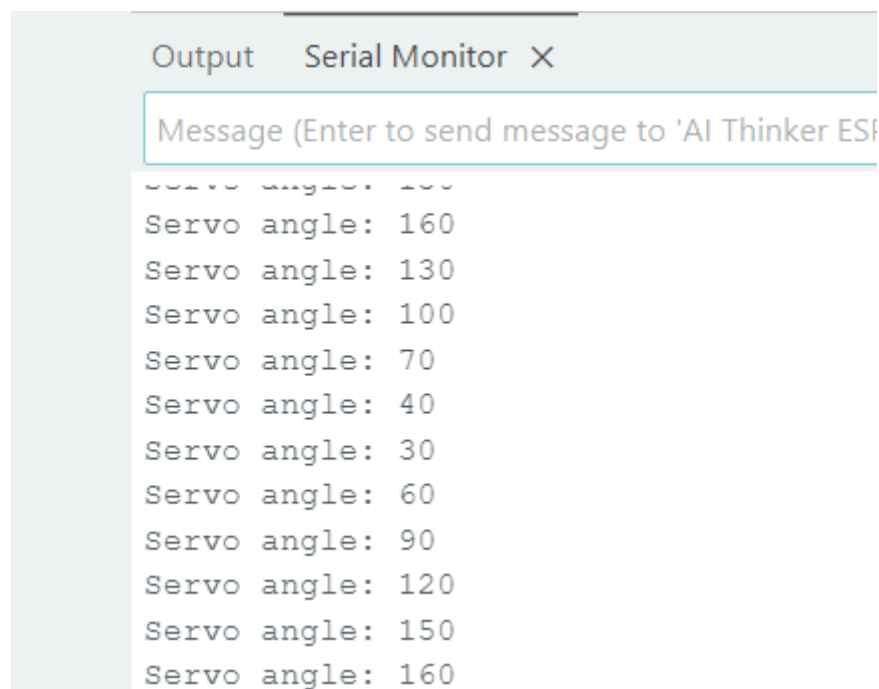


Figure 7.1 Shows the result at serial monitor

7.2 Task 2

Face detection was implemented using the **ESP32 Wrover Module** with **esp32** by **Espressif version 1.0.4**, as the previous attempt using version 3.3.4 on AI Thinker ESP32-CAM did not successfully detect faces. The video stream accurately identified faces within the camera frame, displaying bounding boxes around detected faces in real time.

Detection was reliable under varying lighting conditions and positions, though optimal results were obtained when faces were centrally positioned and well-lit. This demonstrates that the Wrover Module with version 1.0.4 provided better support for built-in face detection algorithms compared to version 3.3.4, highlighting the importance of selecting the appropriate board and software version for specific functionalities.

Overall, the results confirm that both tasks were successfully executed according to their respective objectives, with the board choice and Espressif version playing a key role in enabling the desired features for each task.

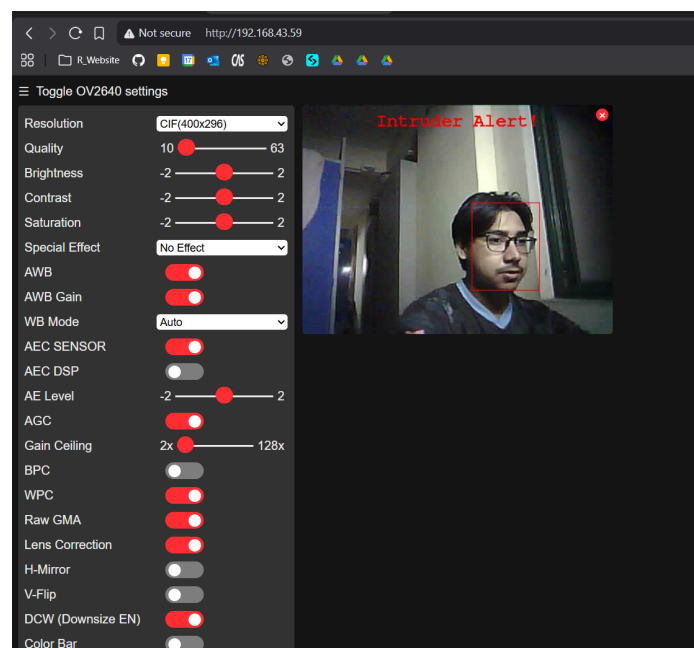


Figure 7.2 Shows the face detector from the IP address CameraWebServer

8. Discussion

The experiment successfully demonstrated the integration of ESP32-CAM modules for both manual servo control and face detection within a smart surveillance system. For **Task 1**, using the **AI Thinker ESP32-CAM** with **Espressif version 3.3.4**, the pushbutton effectively controlled the servo panning, confirming that the system can respond to physical inputs while maintaining a stable video stream. This highlights the capability of the ESP32-CAM to handle concurrent tasks, such as servo actuation and video streaming, without significant lag or performance issues.

In **Task 2**, it became apparent that **face detection and recognition** require specific software and hardware configurations. The team initially attempted to implement face detection using the AI Thinker board with version 3.3.4, but the detection feature failed to perform reliably. After exploring existing resources, the team switched to the **ESP32 Wrover Module** with **version 1.0.4**, which successfully enabled face detection. This outcome demonstrates the importance of selecting the appropriate board and software version to match the intended functionality. The difference in camera modules and microcontroller capabilities between the AI Thinker and Wrover boards also played a critical role in task execution, especially for advanced features like face detection and recognition.

Overall, the lab reinforced key concepts in **mechatronic integration**, showing how microcontrollers, sensors, actuators, and communication interfaces can work together to form a functional surveillance system. It also highlighted practical considerations, such as hardware compatibility and software version selection, which are essential for successful system implementation.

9. Conclusion

The Week 6 Smart Surveillance System lab achieved its objectives by demonstrating the following:

1. **Manual servo control** was successfully implemented with a pushbutton, using the AI Thinker ESP32-CAM and Espressif version 3.3.4, allowing precise horizontal camera movement only when triggered.
2. **Face detection** was effectively achieved using the ESP32 Wrover Module and Espressif version 1.0.4, with the camera accurately detecting and highlighting faces in real time.
3. The experiment emphasized the significance of choosing the correct **ESP32 board and software version** to support specific functionalities, such as servo control versus face detection.
4. The integration of hardware and software components illustrated fundamental principles of **mechatronics**, including sensor feedback, actuator control, and real-time data processing over a network.

In conclusion, the lab provided practical insights into the design and implementation of a small-scale smart surveillance system. It also reinforced the importance of understanding the limitations and capabilities of different microcontroller boards and software versions to ensure successful project outcomes.

10. Recommendations

1. Improve Wi-Fi Stability and Streaming Performance

A stable Wi-Fi connection is crucial for smooth video streaming. Keeping the ESP32-CAM close to the router and minimizing interference can help reduce lag, buffering, and unexpected disconnections during livestreaming.

2. Strengthen Understanding of ESP32-CAM Coding Flow and Configuration Logic

Students often face challenges when editing or troubleshooting ESP32-CAM code due to its more complex structure compared to basic microcontroller sketches. Gaining a clearer understanding of how the code is organized—such as the setup of camera pins, Wi-Fi configuration, and the web server streaming logic—can help students identify errors more easily. Focusing on how different sections interact, rather than specific technical functions, makes it easier to modify and adapt the code without introducing issues.

3. Explore Camera and Streaming Settings Through Hands-On Experimentation

The performance of the ESP32-CAM changes depending on camera settings used during streaming. By experimenting with different resolutions, brightness, and quality levels, students can better understand how each parameter affects clarity, responsiveness, and overall stability. This practical exploration helps students make informed choices when configuring the camera for smoother operation.

11. References

- Random Nerd Tutorials. (n.d.). *How to program / upload code to ESP32-CAM AI-Thinker (Arduino IDE)*. <https://randomnerdtutorials.com/program-upload-code-esp32-cam/>
- Cytron. (n.d.). *CH340 USB to TTL serial cable*.
<https://my.cytron.io/p-ch340-usb-to-ttl-serial-cable>
- Arduino-er. (2020, September). *Program ESP32-CAM using FTDI adapter*.
<https://arduino-er.blogspot.com/2020/09/program-esp32-cam-using-odi-adapter.html>
- Core Electronics. (n.d.). *Use an ESP32-CAM module to stream HD video over local network*.
<https://core-electronics.com.au/guides/esp32-cam-set-up/>
- YouTube. (2020, November 26). *Program ESP32-CAM using FTDI adapter* [Video].
YouTube. <https://www.youtube.com/watch?v=D3MPBPGT3cw>

Appendix A - Circuit Diagram

Task 1

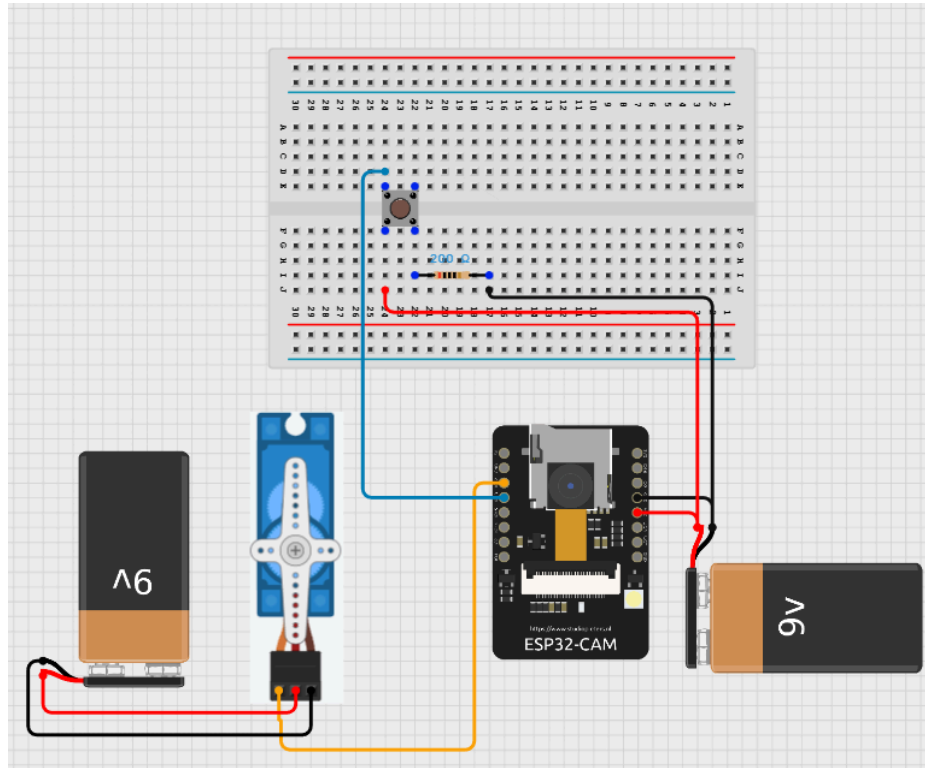


Figure A1 Shows the connection of Task 1 circuit configuration by using Cirkit

Task 2

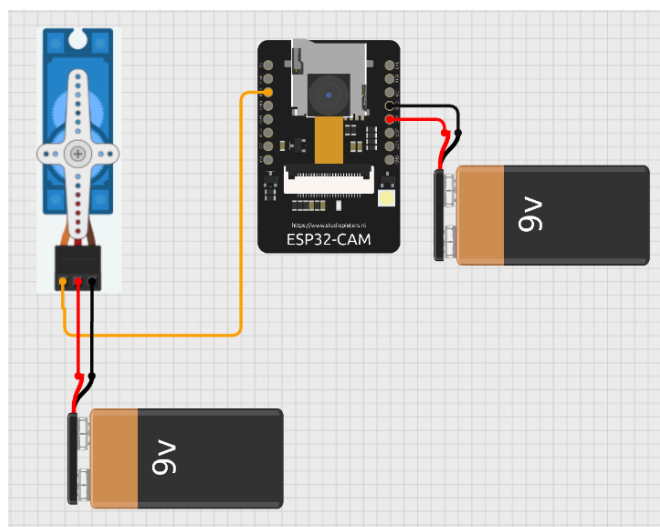


Figure A3 Shows the connection of Task 2 circuit configuration by using Cirkit

Appendix B - Coding Snippets

Task 1

B1 Arduino IDE Coding

```
#include <ESP32Servo.h>
#include "esp_camera.h"
#include <WiFi.h>
#include "board_config.h" // your camera pin definitions
// -----
// WIFI SETTINGS
// -----
const char* ssid = "Jimmy";
const char* password = "jemmeh123";
// -----
// SERVO SETTINGS
// -----
Servo panServo;
int panAngle = 90; // starting angle
const int MIN_ANGLE = 30;
const int MAX_ANGLE = 160;
int stepDirection = 30; // degrees per step
const int SERVO_PIN = 12;
// -----
// BUTTON SETTINGS
// -----
const int BUTTON_PIN = 13;
// -----
// MOTION DETECTION SETTINGS
// -----
const long SHIFT_THRESHOLD = 24000; // sensitivity
const size_t LASTJPEG_MAX = 35000;
static uint8_t lastJPEG[LASTJPEG_MAX];
static size_t lastLen = 0;
bool haveLast = false;
// -----
// CAMERA SERVER
// -----

void startCameraServer();
//-----
```

```

        // Helper: detect movement by sampling differences
between two
        // JPEG buffers. Returns +1 (right), -1 (left), or 0
(no movement).
        //-----
        int detectCameraMovementSampled(const uint8_t* prev,
size_t prevLen,
        const uint8_t* curr, size_t currLen)
        {
            if (!prev || !curr || prevLen == 0 || currLen == 0)
return 0;
            long leftSum = 0;
            long rightSum = 0;
            size_t minLen = (prevLen < currLen) ? prevLen :
currLen;
            const size_t STEP = 300;
            for (size_t i = 0; i + 1 < minLen; i += STEP) {
                int d = (int)curr[i] - (int)prev[i];
                if ((i / STEP) % 2 == 0) leftSum += d;
                else rightSum += d;
            }
            long diff = rightSum - leftSum;
            if (abs(diff) < SHIFT_THRESHOLD) return 0;
            return (diff > 0) ? +1 : -1;
        }
        // -----
        // SETUP
        // -----
        void setup() {
            Serial.begin(115200);
            delay(100);
            // attach servo
            panServo.attach(SERVO_PIN);
            panServo.write(panAngle);
            // setup button
            pinMode(BUTTON_PIN, INPUT_PULLUP);
            // camera config
            camera_config_t config;
            config.pin_pwdn = PWDN_GPIO_NUM;
            config.pin_reset = RESET_GPIO_NUM;
            config.pin_xclk = XCLK_GPIO_NUM;
            config.pin_sscb_sda = SIOD_GPIO_NUM;

```

```

config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.ledc_timer = LEDC_TIMER_0;
config.ledc_channel = LEDC_CHANNEL_0;
config.pixel_format = PIXFORMAT_JPEG;
config.frame_size = FRAMESIZE_QVGA;
config.jpeg_quality = 15;
config.fb_count = 2;
if (esp_camera_init(&config) != ESP_OK) {
    Serial.println("Camera init failed!");
    while (true) { delay(1000); }
}
// connect WiFi
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
unsigned long t0 = millis();
while (WiFi.status() != WL_CONNECTED) {
    if (millis() - t0 > 20000) break;
    Serial.print(".");
    delay(300);
}
Serial.println();
if (WiFi.status() == WL_CONNECTED) {
    Serial.print("WiFi connected. Stream: http://");
    Serial.println(WiFi.localIP());
}
startCameraServer();
}

// -----
// MAIN LOOP

```

```

// -----
void loop() {
    bool buttonPressed = (digitalRead(BUTTON_PIN) ==
LOW);

    camera_fb_t* fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("FRAME ERROR");
        delay(150);
        return;
    }
    int direction = 0;
    if (haveLast) {
        direction = detectCameraMovementSampled(lastJPEG,
lastLen, fb->buf, fb->len);
    }
    // store JPEG for next comparison
    size_t copyLen = fb->len;
    if (copyLen > LASTJPEG_MAX) copyLen = LASTJPEG_MAX;
    memcpy(lastJPEG, fb->buf, copyLen);
    lastLen = copyLen;
    haveLast = true;
    esp_camera_fb_return(fb);
    // move servo if button pressed OR motion detected
    if (buttonPressed || direction != 0) {
        if (direction == +1) stepDirection = 5;
        else if (direction == -1) stepDirection = -5;
        panAngle += stepDirection;
        // reverse at limits
        if (panAngle >= MAX_ANGLE) { panAngle = MAX_ANGLE;
stepDirection = -stepDirection; }
        if (panAngle <= MIN_ANGLE) { panAngle = MIN_ANGLE;
stepDirection = -stepDirection; }
        panServo.write(panAngle);
        Serial.print("Servo angle: ");
Serial.println(panAngle);
    }
    delay(120); // smooth motion
}

```

Task 2

B2 Arduino IDE Coding

```
CameraWebServer.ino

#include "esp_camera.h"
#include <WiFi.h>

//
// WARNING!!! Make sure that you have either selected ESP32
// Wrover Module,
// or another board which has PSRAM enabled
//

// Select camera model
// #define CAMERA_MODEL_WROVER_KIT
// #define CAMERA_MODEL_ESP_EYE
// #define CAMERA_MODEL_M5STACK_PSRAM
// #define CAMERA_MODEL_M5STACK_WIDE
#define CAMERA_MODEL_AI_THINKER

#include "camera_pins.h"

const char* ssid = "Jimmy";
const char* password = "jemmeh123";

void startCameraServer();

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Serial.println();

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM;
    config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM;
    config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM;
```

```

config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
//init with high specs to pre-allocate larger buffers
if(psramFound()){
    config.frame_size = FRAMESIZE_UXGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}

#ifdef CAMERA_MODEL_ESP_EYE
    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

// camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
}

sensor_t * s = esp_camera_sensor_get();
//initial sensors are flipped vertically and colors are a
bit saturated
if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); //flip it back
    s->set_brightness(s, 1); //up the blightness just a bit

```

```
s->set_saturation(s, -2); //lower the saturation
}
//drop down frame size for higher initial frame rate
s->set_framesize(s, FRAMESIZE_QVGA);

#if defined(CAMERA_MODEL_M5STACK_WIDE)
s->set_vflip(s, 1);
s->set_hmirror(s, 1);
#endif

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("");
Serial.println("WiFi connected");

startCameraServer();

Serial.print("Camera Ready! Use 'http://");
Serial.print(WiFi.localIP());
Serial.println("' to connect");
}

void loop() {
    // put your main code here, to run repeatedly:
    delay(10000);
}
```


Appendix C - Results

Task 1

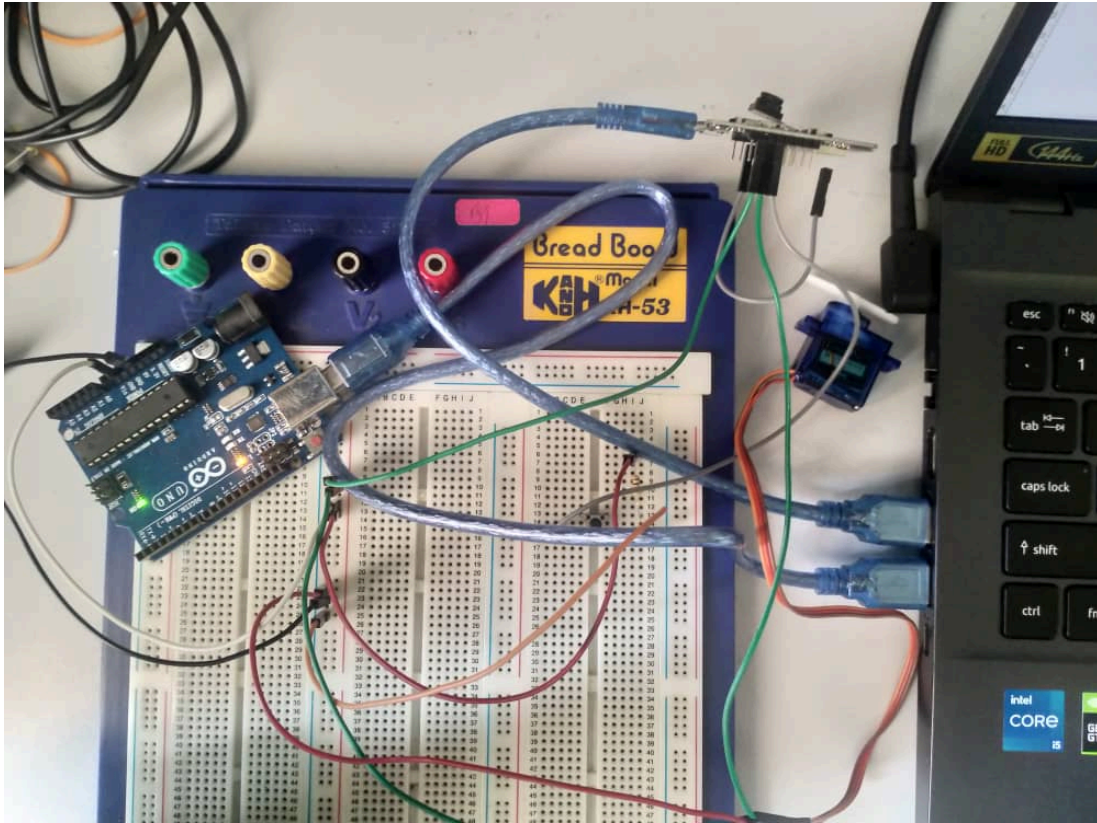


Figure C1 Shows the snap of Task 1

Task 2

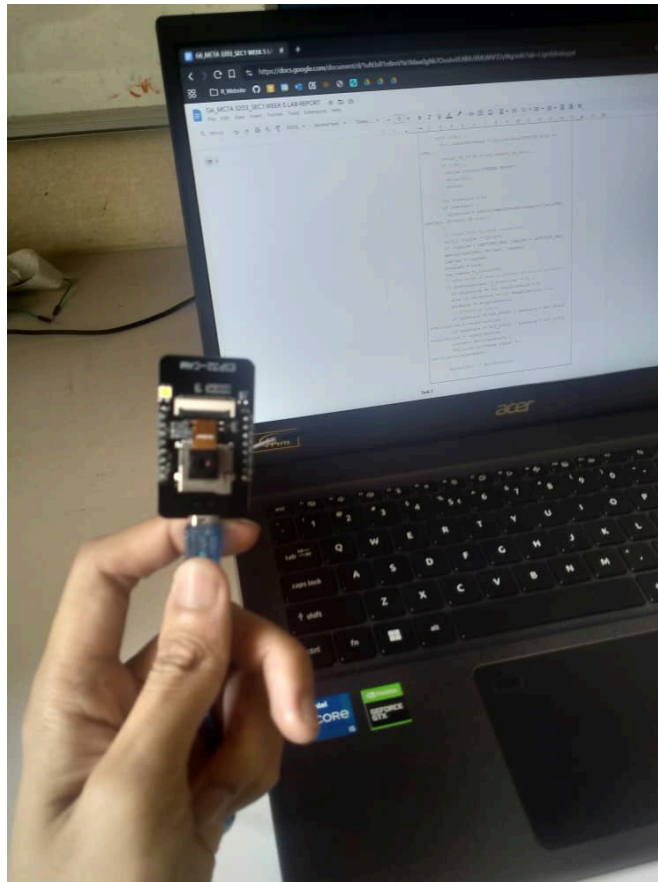


Figure C2 Shows the ESP32-CAM

Acknowledgments

We would like to express our sincere gratitude to our lecturer and lab instructor for their continuous guidance, clear explanations, and assistance with experimental setups throughout this experiment. Their detailed instructions, along with the necessary files provided, helped us understand how to properly set up the Arduino UNO microcontroller, PyCharm, and serial communication, and gave us the confidence to carry out the tasks effectively. We also extend our heartfelt appreciation to the lab assistants, whose patience and support in setting up the equipment, troubleshooting circuits, and clarifying procedural steps were instrumental in the smooth progress of the experiment.

Special thanks are also due to our team members, who contributed significantly to every stage of this project, including circuit assembly, coding, data collection, analysis, and documentation. Their collaboration, dedication, and teamwork ensured that each task was completed accurately and efficiently. We also sincerely appreciate our classmates and lab partners for their cooperation, ideas, and teamwork during the experiment. Together, we troubleshooted connection issues, shared components, and tested the code to ensure everything functioned as expected.

Overall, the success of this experiment would not have been possible without the guidance and support of both our instructors and lab partners, and we are truly grateful for their contributions.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specific in references and acknowledgement, and that the original work contained herein have not been untaken or done by unspecified sources or a person.

We hereby certify that this report has **not been done by only one individual** and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate.

We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report.

We, therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature: 

Name: Muhammad Fahim Bin Ahmad Norisham

Matric Number: 2312369

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Zamarul Azim Bin Zulkhibri

Matric Number: 2312451

Read /

Understand /

Agree /

Signature: 

Name: Muhammad Hazimuddin Bin Fairuz

Matric Number: 2312479

Read /

Understand /

Agree /